SupaClock – `test_general` (Demo)

Documento de repaso para la demostracion. Cubre el firmware corriendo en el entorno `test_general` (src/tests/test_general.c), que es la integracion completa de todos los sensores, GUI, BLE y power-management sobre un ESP32-C3.

---

1. Resumen ejecutivo

`test_general` enciende **todos** los subsistemas a la vez:

- **Display ST7789 240x280** con LVGL – UI con 5 pantallas ciclicas + 2 sub-pantallas de configuracion.
- **2 botones GPIO** (NEXT y SELECT) con deteccion short/long press.
- **5 sensores I2C/ADC**: BMI160 (IMU), MAX30102 (HR/SpO2), MAX30205 (temp), MAX17048 (fuel gauge), AD8232 (ECG por ADC continuo).
- **BLE NimBLE** con 4 caracteristicas GATT (IMU, telemetria agregada, ECG streaming, comandos RX).
- **3 modos de energia** (SPORT/NORMAL/SAVER) que cambian cadencias de todos los sensores y del flush BLE; persistidos en NVS.
- **Light sleep dinamico** + PM locks para los caminos criticos.

El programa corre 7 tareas FreeRTOS coordinadas con dos mutexes.

---

2. Mapa de hardware (ESP32-C3)

Periferico	Pin / Bus	Notas
I2C bus	configurado en `i2c_bus.c`	sensores: BMI160, MAX30102, MAX30205, MAX17048
ST7789 SPI	configurado en `st7789.c`	240x280, backlight por PWM
AD8232 OUT	ADC1 CH0 = **GPIO0**	modo continuous DMA
AD8232 SDN	GPIO 2	encendido/apagado del front-end
Boton NEXT	GPIO 10	pull-up interno, activo en bajo
Boton SELECT	GPIO 1	pull-up + wakeup deep-sleep

---

3. Arquitectura del firmware

3.1 Datos compartidos

Una sola estructura agrupa todas las lecturas; protegida por `xSensorDataMutex`:

```
typedef struct {
    int16_t  ax, ay, az, gx, gy, gz;      // IMU 6-DOF raw
    uint32_t steps_sw;                    // contador de pasos por SW
    float    temperature_c;
    uint16_t battery_mv;
    float    battery_soc;
    uint8_t  hr_bpm, spo2_pct;
    bool     finger_present;
    uint32_t hr_updated_ms, spo2_updated_ms,
             temp_updated_ms, bat_updated_ms;
} shared_sensor_data_t;
```

`xGuiSemaphore` protege a LVGL (no es thread-safe).

3.2 Tareas (FreeRTOS)

Todas se crean al final de `app_main`:

Task	Prio	Stack	Frecuencia	Rol
`gui_task`	5	4 KB	30 Hz on / 10 Hz off	dibuja LVGL, lee botones, auto-off
`imu_task`	6	4 KB	50/25/12.5 Hz (modo)	BMI160 -> pasos + jerk + BLE IMU
`hrm_task`	5	4 KB	poll 10 Hz	MAX30102 (continuo o spot por modo)
`system_task`	3	4 KB	cada 2 s	temperatura + bateria
`ble_tx_task`	4	4 KB	flush 1/10/60 s	flush del buffer agregado
`ecg_task`	7	4 KB	bursts del DMA	AD8232 -> BLE 0xFF03
`perf_monitor_task`	2	6 KB	cada 10 s	logs de heap, CPU, locks

`ecg_task` es la **mas prioritaria** (7) porque el DMA del ADC no espera; si no se drena rapido, hay overruns.

3.3 Flujo general

```
              +----------+
I2C   ---> | sensors  | --> sensor_data (mutex)
              +----------+            |
                                       v
              +----------+         +---------+
ADC  --->  | ecg_task | --BLE-> | cliente |
              +----------+         +---------+
                                        ^
              +----------+               |
GPIO --->  | gui_task | -- LVGL --> ST7789
              +----------+
```

---

4. Modos de energia

Definidos en lib/power_modes/power_modes.c. El modo activo se persiste en NVS bajo `supaclock/power_mode`.

Cadencia	SPORT	NORMAL	SAVER
HRM polling (FIFO)	100 ms (continuo)	100 ms (spot c/10 min)	100 ms (spot c/30 min)
SpO2 automatico	cada 5 min	cada 30 min	solo manual
HRM SHDN entre medidas	no	si	si
IMU polling	20 ms (50 Hz)	40 ms (25 Hz)	80 ms (12.5 Hz)
MAX30205 temp	30 s	5 min	15 min
MAX17048 bateria	30 s	30 s	30 s
BLE agg flush	1 s	10 s	60 s
Display auto-off default	30 s	15 s	8 s

Las tasks consultan `power_get_profile()` al inicio de cada iteracion, asi un cambio de modo se aplica al proximo ciclo sin notificacion explicita.

---

5. Display + UI (LVGL)

5.1 Pantallas

```
typedef enum {
    SCREEN_HOME = 0,   // ciclables -+
```

```
    SCREEN_BIO,        //            |
    SCREEN_HRSPOT,     //            | NEXT_SHORT recorre
    SCREEN_ECG,        //            |
    SCREEN_MENU,       //            +
    SCREEN_MODE,       // sub-screen, solo desde MENU
    SCREEN_SETTINGS,   // sub-screen, solo desde MENU
} ui_screen_t;
```

Build de cada pantalla: `build_home()`, `build_bio()`, `build_hrspot()`, `build_ecg()`, `build_menu()`, `build_mode()`, `build_settings()`. Todas se construyen una sola vez en `build_ui()`.

5.2 HOME

Reloj grande (mm:ss desde boot), linea con el modo activo, y 4 *cards*: pasos, **arco de bateria** (LVGL arc + label central), HR y actividad (activo/reposo segun magnitud del acelerometro).

5.3 BIO

HR, SpO2, temperatura con *aging* relativo (`ahora`, `hace 12 s`, `hace 5 m`) y un texto de "Estado" por color segun el HR.

5.4 HRSPOT

Maquina de estados del SPOT del MAX30102: `IDLE -> SETTLING (5 s) -> MEASURING -> DONE / FAILED / ABORTED`. Mostramos progreso porcentual, resultado y calidad (BUENA/REGULAR/POBRE). SELECT inicia/cancela la medicion.

5.5 ECG

- Estado **stop** -> instrucciones.
- Estado **rec** -> cronometro mm:ss y badge "o REC".
- SELECT toggle del modo ECG (tambien lo puede activar el cliente BLE escribiendo `0x01` a la char de comandos 0xFF04).

5.6 MENU + sub-pantallas

Items del menu principal:

```
1. Modo Energia        -> entra a MODE
2. Auto-off Pant.      -> entra a SETTINGS
3. Reiniciar Pasos     -> steps_sw = 0
4. Vincular BLE        -> re-advertise (ya esta activo siempre)
5. Apagar              -> esp_deep_sleep_start, wake por SELECT
6. Tx IMU: ON/OFF      -> toggle del envio IMU por BLE
7. Reset Bateria       -> max17048_reset() (POR + Quick Start)
```

5.7 Botones – eventos

Evento	En pantallas ciclables	En MENU/MODE/SETTINGS	En ECG / HRSPOT
NEXT short	siguiente pantalla	mover seleccion	–
NEXT long	pantalla anterior	volver a MENU	pantalla anterior
SELECT short	–	ejecutar/aplicar	start/stop/abort
SELECT long	volver a HOME	volver a HOME	volver a HOME

5.8 Backlight & auto-off

`gui_task` cuenta inactividad por *decimas de frame* (~33 ms cada uno). Al pasar `power_get_display_off_s(modo)` segundos sin boton, baja el PWM del ST7789 a 0. Cualquier boton lo reenciende **sin disparar la accion** del boton en si (pulsacion de "wake-up" no cuenta como input).

Cuando la pantalla esta apagada, `gui_task` no ejecuta `lv_timer_handler()` y baja a 10 Hz para ahorrar CPU.

---

6. Sensores

6.1 BMI160 – IMU + pasos + jerk

`imu_task` lee accel + giro a la cadencia del modo. Por iteracion:

1. Lee 6 int16 (`bmi160_read_accel_gyro`).
2. Calcula **jerk** (`|?accel|2` escalado a 0-255) y se lo pasa al MAX30102 (`max30102_set_motion_level`) – sirve de gate del HRM ante movimiento.
3. Llama al **algoritmo de pasos por SW** (`step_algo_update`), que detecta picos de magnitud sobre un umbral con histeresis y refractario.
4. Si esta habilitado (item 6 del menu), envia 12 B raw por BLE 0xFF01.
5. Actualiza `sensor_data` (mutex).

   El step counter HW del BMI160 esta deshabilitado a proposito (no ira en produccion). Solo se usa el algoritmo SW.

6.2 MAX30102 – HR/SpO2

Driver con dos modos de operacion:

- **Continuous** (solo SPORT): el sensor esta siempre encendido, el FIFO se drena cada 100 ms, se publican TLV de HR cuando hay valor estable.
- **Spot** (NORMAL/SAVER): el sensor esta apagado (`max30102_shutdown`), arranca cada `hrm_auto_period_ms`, hace SETTLING (5 s descartando) y luego MEASURING. Cuando termina, publica un TLV `SPOT_RESULT` y vuelve a apagarse.

El usuario puede iniciar un SPOT manual desde la pantalla HRSPOT independiente del modo.

6.3 MAX30205 – temperatura

`system_task` lo lee cada `temp_period_ms` con `max30205_read_temperature`. Publica TLV `TEMP` (i16 x100 ?C).

6.4 MAX17048 – fuel gauge

Driver: lib/max17048_driver/max17048.c. Lee VCELL (mV) y SOC (% con fraccion). Mejoras introducidas:

- **Deteccion de POR** en init: si el chip arranco desde frio (bit RI=1 del registro STATUS), ejecuta Quick Start para que la primera lectura parta del voltaje actual y no de un valor arbitrario.
- **Hibernate configurado** (HIBRT 0x80/0x30): bajo cargas pequenas el ADC interno baja de 250 ms a 45 s, lo que reduce el ruido del SOC.
- **Reset por software** (`max17048_reset`) expuesto en el menu #7 – manda el comando POR (0x5400 -> CMD reg 0xFE), encadena Quick Start y limpia el bit POR. Pensado para cuando se cambia la celda.
- **VRESET configurado** (umbral de deteccion "bateria removida").

`system_task` publica TLV `BAT` cada 30 s (en todos los modos).

6.5 AD8232 – ECG (ADC continuous)

Front-end analogico con salida unipolar. La senal entra por GPIO0 (ADC1 CH0). El driver del ADC del IDF se configura en modo **continuous DMA**:

- Frecuencia HW: **20 kHz** (minimo en C3).
- Frame DMA: 1024 B -> 256 muestras (`adc_digi_output_data_t` = 4 B).
- Buffer interno del DMA: 4096 B.
- Cada frame se notifica cada 256 / 20 000 ~ **12.8 ms**.

`ad8232_init_dma` configura GPIOs (SDN, LO+/LO-) y prepara el DMA; `ad8232_start_dma` / `ad8232_stop_dma` lo arrancan/detienen on-demand.

El detalle del flujo ECG esta en la seccion 8.

---

7. Bluetooth (NimBLE)

Implementacion: lib/ble_telemetry/ble_telemetry.c.

7.1 Servicio y caracteristicas

Un unico servicio primario `0xFF00` con cuatro caracteristicas:

UUID	Dir	Tipo	Contenido
`0xFF01` IMU	Read + Notify	binario	12 B = 6 x i16 (ax,ay,az,gx,gy,gz) raw
`0xFF02` AGG	Read + Notify	TLV agregado	header + records (ver ?7.3)
`0xFF03` ECG	Read + Notify	streaming	chunks de int16 (10 muestras = 20 B)
`0xFF04` CMD	Write	comandos RX	1 B: 0x01=start ECG, 0x00=stop ECG

El advertising publica **`SupaClock_BLE`** con appearance `0x00C1` (watch). Intervalo de advertising: **1000 ms** (deliberadamente lento para permitir light sleep entre eventos).

Sin pairing/bonding (compatibilidad con BlueZ y tests).

7.2 Camino directo vs. agregado

Hay dos maneras de mandar datos por BLE:

Directo – `ble_telemetry_send_imu()` y `ble_telemetry_send_ecg()`. Llaman a `ble_gatts_notify_custom` inmediatamente. Se usan para flujos de alta cadencia que no toleran agregacion: IMU (50 Hz) y ECG (500 Hz).

Agregado – `ble_tx_push()` apila records TLV en un buffer (`s_agg_buf`, max 200 B). Cuando `ble_tx_task` llama a `ble_tx_flush()` segun la cadencia del modo (1/10/60 s), emite **un solo notify** con un header + todos los records pendientes. Esto agrupa senales lentas (HR, SpO2, temp, bateria, eventos) y reduce drasticamente el costo energetico de la radio.

Si llega una senal de prioridad alta (resultado SPOT, cambio de modo) se puede pasar `flush_now_mode != 0xFF` a `ble_tx_push()` para forzar el flush inmediato sin esperar al timer.

7.3 Formato del paquete agregado (0xFF02)

```c
typedef struct __attribute__((packed)) {
    uint32_t boot_ts_ms;   // ms desde boot del primer record en el buffer
    uint8_t  power_mode;   // 0=SPORT 1=NORMAL 2=SAVER (al momento del flush)
    uint8_t  payload_len;  // bytes de records TLV que siguen
} ble_agg_header_t;

// payload: secuencia de records TLV
//   uint8_t  type;
```

```
//   uint8_t  len;
//   uint8_t  data[len];
```

Tipos definidos:

Tipo	Bytes	Contenido
`0x01 HR`	4	u16 delta_ms, u8 bpm, u8 quality
`0x02 SPO2`	4	u16 delta_ms, u8 pct, u8 quality
`0x03 TEMP`	4	u16 delta_ms, i16 temp_x100
`0x04 BAT`	5	u16 delta_ms, u16 mv, u8 soc
`0x05 STEPS`	4	u32 total_steps
`0x06 MODE_EVT`	1	u8 nuevo modo
`0x07 SPOT_RESULT`	6	u8 bpm, u8 spo2, u16 dur_ms, u8 quality, u8 aborted

`delta_ms` se calcula relativo a `boot_ts_ms` del header -> el cliente reconstruye el timestamp absoluto sin necesidad de un reloj sincronizado.

7.4 Modo ECG (mutex implicito vs. agregado)

Cuando `ecg_mode_active = true`:

- `ble_telemetry_send_imu` se vuelve no-op (no compite con el ECG por la banda de la radio).
- `agg_emit_locked` descarta el contenido del buffer agregado.

Esto evita saturar la radio durante la grabacion de ECG, que ya domina el trafico BLE.

---

8. ECG en detalle

`ecg_task` (test_general.c:805):

```
HW continuous DMA @ 20 kHz
   |
   | 256 muestras (4B x 256 = 1024 B) por frame  ->  cada ~12.8 ms
   v
adc_continuous_read(dma_buf, 1024, ...)
   |
   | por cada muestra del frame:
```

```
   |   sum += raw;  count++;
   |   if (count == 40) {
   |       chunk[i++] = sum / 40;     // promedio = boxcar antialias
   |       sum = 0; count = 0;
   |       if (i == 10) {
   |           ble_send_ecg(chunk);   // 10 x i16 = 20 B
   |       }
   |   }
   v
500 Hz efectivos al BLE  ->  cliente plotea
```

- **Decimacion 1:40 con promedio**: 20 kHz -> 500 Hz. El boxcar (sum/40) actua como filtro paso-bajo barato antes de bajar la frecuencia, y evita aliasing de armonicos altos.
- **Chunks de 10 int16**: ~20 ms de senal por notify, balance entre latencia y overhead BLE.

8.1 PM lock

`light_sleep_enable=true` esta activo globalmente. Si el sistema se duerme durante el DMA, al despertar el reloj del APB se reconfigura y la senal del ADC muestra **escalones cuadrados** periodicos.

Solucion: tomar un PM lock del tipo `ESP_PM_NO_LIGHT_SLEEP` mientras el ADC continuo esta corriendo:

```
esp_pm_lock_create(ESP_PM_NO_LIGHT_SLEEP, 0, "ecg", &s_ecg_pm_lock);

// al iniciar:
esp_pm_lock_acquire(s_ecg_pm_lock);
ad8232_start_dma();

// al detener:
ad8232_stop_dma();
esp_pm_lock_release(s_ecg_pm_lock);
```

El lock solo bloquea el light sleep – la radio sigue gestionando su propio sleep entre intervals de conexion, y el DFS sigue libre.

---

9. Power management

9.1 Light sleep dinamico

```
esp_pm_config_esp32c3_t pm_config = {
    .max_freq_mhz = 160,
    .min_freq_mhz = 10,
    .light_sleep_enable = true,
```

```
};
esp_pm_configure(&pm_config);
```

Esto activa **DFS (Dynamic Frequency Scaling)**: la CPU baja a 10 MHz cuando esta idle y entra a light sleep cuando todas las tasks estan bloqueadas. Drivers que necesitan reloj alto toman su propio lock (I2C, SPI, ADC, NimBLE).

9.2 PM locks usados

- **ECG** (`ESP_PM_NO_LIGHT_SLEEP`) – solo durante grabacion.
- Internos del ADC continuo (APB max) – mientras el DMA corre.
- Internos de NimBLE – durante eventos de conexion.

`perf_monitor_task` puede dumpear los locks activos con `esp_pm_dump_locks(stdout)` (necesita `CONFIG_PM_PROFILING`).

9.3 Deep sleep

El item "Apagar" del menu entra a **deep sleep** con wakeup por GPIO (SELECT). Todo el RAM se pierde; el contador de pasos del BMI160 SW tambien. Al despertar, vuelve a `app_main`.

---

10. Persistencia (NVS)

Namespace: `supaclock`. Claves:

Clave	Tipo	Contenido
`power_mode`	u8	modo activo (0=SPORT, 1=NORMAL, 2=SAVER)
`off_sport_s`	u16	auto-off pantalla en SPORT
`off_normal_s`	u16	auto-off pantalla en NORMAL
`off_saver_s`	u16	auto-off pantalla en SAVER

Nada mas se persiste – pasos, bateria, etc. se reinician en cada boot.

---

11. Pantalla por pantalla – guion de demo

Paso	Accion	Mostrar
1	Encender (boot logs)	Deteccion de cada sensor en monitor serial
2	HOME	Reloj, modo SPORT, arco de bateria real

Paso	Accion	Mostrar
3	NEXT_SHORT -> BIO	HR/SpO2/temp con timestamps relativos
4	NEXT_SHORT -> HRSPOT	SELECT inicia spot, ver progreso -> resultado
5	NEXT_SHORT -> ECG	SELECT empieza grabacion, ver el ECG en el plotter Python
6	NEXT_SHORT -> MENU	navegar items con NEXT, ejecutar con SELECT
7	Modo Energia -> SPORT/NORMAL/SAVER	Cambian las cadencias, log lo confirma
8	Auto-off Pant.	Cambiar valor de auto-off del modo activo
9	Reset Bateria	El SOC se recalcula desde el voltaje actual
10	Apagar	Deep sleep; SELECT despierta de nuevo

Cliente BLE recomendado: `tools/supaclock_monitor.py` (Python + bleak). Se conecta a `SupaClock_BLE`, recibe los notify de las 4 caracteristicas y plotea telemetria + ECG en vivo. Los CSV de ECG se guardan en `tools/supaclock_ecg_*.csv`.

---

12. Glosario rapido de archivos

Archivo	Que hace
src/tests/test_general.c	El demo completo (este documento)
lib/ble_telemetry/	Stack BLE NimBLE + GATT + buffer agregado
lib/power_modes/	Tabla de perfiles + persistencia NVS
lib/ad8232_driver/	ADC continuous + GPIO control del front-end
lib/max17048_driver/	Fuel gauge – POR, Quick Start, hibernate
lib/max30102_driver/	HR/SpO2 con SM de SPOT
lib/max30205_driver/	Sensor de temperatura I2C
lib/bmi160_driver/	Acelerometro + giroscopio
lib/step_algorithm/	Pedometro por SW (peak detection)
lib/st7789_driver/	Display SPI + brightness PWM

Archivo	Que hace
lib/gpio_buttons/	Debounce + short/long press
lib/i2c_bus/	Bus compartido para los 4 sensores I2C

---

13. Cosas a destacar en la demo

1. **Una sola tarea por dominio** – clarisima separacion de responsabilidades.
2. **Doble camino BLE** (directo + agregado) – optimizacion energetica real y medible.
3. **Modos de energia declarativos** – toda la cadencia en una tabla; las tasks la consultan, no requiere if/switch ni callbacks.
4. **PM lock para ECG** – solucion correcta al problema del light sleep durante DMA, sin sacrificar ahorro energetico en el resto del tiempo.
5. **MAX17048 con POR detection + hibernate** – fuel gauge "industrial", no hace reset destructivo en cada boot pero si responde a una celda nueva.
6. **TLV con timestamp diferencial** – cliente reconstruye timeline sin necesidad de reloj sincronizado, ahorra bytes en cada record.